Joins With Clauses

==========

JOIN + GROUP BY

Q1. Count employees per department.

```
SELECT d.department_name, COUNT(e.emp_id) AS Total_Employees
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

department_name	Total_Employees
Engineering	3
Finance	1
Human Resources	1
Marketing	2
Operations	1

Q2. Total salary per department.

```
SELECT d.department_name, SUM(e.salary) AS Total_Salary
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

department_name	Total_Salary
Engineering	199000
Finance	55000

Human Resources	50000
Marketing	105000
Operations	48000

Q3. Number of projects per employee.

```
SELECT e.name, COUNT(p.project_id) AS Total_Projects
FROM Employees e
LEFT JOIN Projects p ON e.emp_id = p.emp_id
GROUP BY e.name;
```

name	Total_Projects
Arjun	0
Bhavana	2
Charan	1
Divya	1
Eswar	0
Farhan	1
Gowri	1
Harish	0
Ishita	0
Jagan	1

JOIN + ORDER BY

Q1. Employees with department names, ordered by salary (highest first).

```
SELECT e.name, d.department_name, e.salary
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id
ORDER BY e.salary DESC;
```

name	department_name	salary
Charan	Engineering	72000
Bhavana	Engineering	65000
Gowri	Engineering	62000
Jagan	Marketing	60000
Farhan	Finance	55000
Arjun	Human Resources	50000
Harish	Operations	48000
Divya	Marketing	45000

(Eswar/Ishita excluded — plain JOIN behaves as INNER JOIN.)

Q2. Employee names and project names, ordered by employee name.

```
SELECT e.name, p.project_name
FROM Employees e
JOIN Projects p ON e.emp_id = p.emp_id
ORDER BY e.name;
```

name	project_name
Bhavana	Website Redesign

Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign
Farhan	Payroll System
Gowri	Backend Optimization
Jagan	Customer Analytics

Q3. Departments and employee count, ordered by count (highest first).

```
SELECT d.department_name, COUNT(e.emp_id) AS Employee_Count
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY Employee_Count DESC;
```

department_name	Employee_Count
Engineering	3
Marketing	2
Operations	1
Human Resources	1
Finance	1

JOIN + GROUP BY + ORDER BY + HAVING

Q1. Departments with more than 1 employee, ordered by count.

```
SELECT d.department_name, COUNT(e.emp_id) AS Employee_Count
```

```
FROM Departments d
JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING COUNT(e.emp_id) > 1
ORDER BY Employee_Count DESC;
```

department_name	Employee_Count
Engineering	3
Marketing	2

Q2. Employees with more than 1 project, ordered by count.

```
SELECT e.name, COUNT(p.project_id) AS Project_Count
FROM Employees e
JOIN Projects p ON e.emp_id = p.emp_id
GROUP BY e.name
HAVING COUNT(p.project_id) > 1
ORDER BY Project_Count DESC;
```

name	Project_Count
Bhavana	2

Q3. Departments with total salary > 50000, ordered by total salary.

```
SELECT d.department_name, SUM(e.salary) AS Total_Salary
FROM Departments d
JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING SUM(e.salary) > 50000
```

ORDER BY Total_Salary DESC;

department_name	Total_Salary
Engineering	199000
Marketing	105000
Finance	55000

(Human Resources = 50000 and Operations = 48000 are correctly excluded — neither is strictly greater than 50000.)

Queries and SubQueries

==========

What is Subquery

==========

a subquery is a query inside another query.

it is written inside parenthesis() and its result is used by the outer query.

it is also called.

inner Query

Nested Query

inner select.

general syntax

---

select column-list

from table

where column OPERATOR(SELECT COLUMN FROM TABLE WHERE CONDITION)

purpose of subquery

---

to break down complex problems into smaller queries.

to use the result of one query as input to another

to avoid multiple steps(write in a single Query insted of temp tables)

types of subqueries

---

1. Single-row-subquery

2. multi-row subquery

3. nested subquery.

4. Correlated subquery.

5. Exists/not Exists subquery

1. Single-row-subquery

---

1. Single Row Subqueries

Single row subqueries return exactly one row and one column. They use comparison operators like =, >, <, >=, <=, !=.

Characteristics:

- **Returns only one value**
- **Can use single-value comparison operators**
- **If subquery returns no rows or multiple rows, it causes an error**

Syntax:

SELECT columns

FROM table

WHERE column operator (SELECT column FROM table WHERE condition);

2. Multi-row Subquery (IN)

—----------

1. Multi-Row Subqueries with IN

A Multi-Row Subquery returns more than one row. The IN operator is used when you want to compare a value with multiple values returned by the subquery.

Characteristics

- **Returns multiple rows**
- **Uses the IN operator**
- **Checks whether a value exists in the list returned by the subquery**
- **Equivalent to multiple OR conditions**
- **Most commonly used for matching values**

Syntax

```
SELECT columns
FROM table
WHERE column IN
   (SELECT column
    FROM table
    WHERE condition);
```

3. Multi-row Subquery (ANY / SOME)

—----------

Multi row subqueries return multiple rows. When using ANY or SOME (they are synonymous), the condition is true if it matches at least one value returned by the subquery.

Characteristics:

- **Returns multiple values**
- **ANY and SOME are interchangeable**
- **Condition is true if ANY of the values satisfy the condition**

- **Often used with IN, >ANY, <ANY, =ANY, etc.**

Syntax:

SELECT columns

FROM table

WHERE column operator ANY/SOME (SELECT column FROM table WHERE condition);

4. Multi-row Subquery (ALL)

—----------

The ALL operator means the condition must be true for ALL values returned by the subquery.

Characteristics:

- **Condition must be true for ALL values in the subquery result**
- **Often used with >ALL, <ALL, >=ALL, <=ALL**
- **>ALL means greater than the maximum value**
- **<ALL means less than the minimum value**

Syntax:

SELECT columns

FROM table

WHERE column operator ALL (SELECT column FROM table WHERE condition);

5. Nested Subquery

—----------

Nested subqueries are subqueries within subqueries. They can be multiple levels deep and are executed from the innermost to the outermost.

Characteristics:

- **Multiple levels of nesting**
- **Inner queries execute first**
- **Can combine different types of subqueries**
- **Useful for complex conditions**

Syntax:

```
SELECT column_list
FROM table
WHERE column = (SELECT column
        FROM table
        WHERE column = (SELECT column
                FROM table
                WHERE condition));
```

6. Correlated Subquery

—----------

Correlated subqueries are subqueries that reference columns from the outer query. Unlike regular subqueries that execute once, correlated subqueries execute once for each row processed by the outer query.

Characteristics:

- **References columns from the outer query**
- **Executes multiple times (once per outer row)**
- **Cannot be executed independently**
- **Often used with EXISTS, NOT EXISTS**
- **Generally slower than non-correlated subqueries**

Syntax

```
SELECT columns
FROM table1 outer_alias
WHERE condition (SELECT column
            FROM table2 inner_alias
        WHERE inner_alias.column = outer_alias.column);
```

7. EXISTS / NOT EXISTS Subquery

—----------

EXISTS and NOT EXISTS

EXISTS is used to test whether a subquery returns any rows. It returns TRUE if the subquery returns at least one row, FALSE otherwise. NOT EXISTS returns TRUE if the subquery returns no rows.

Characteristics:

- **Returns TRUE/FALSE (boolean result)**
- **More efficient than IN for large datasets**
- **Often used with correlated subqueries**
- **Stops execution as soon as first match is found**
- **SELECT clause in EXISTS doesn't matter (often use SELECT 1)**

Syntax:

```
SELECT columns
FROM table1
WHERE EXISTS (SELECT 1 FROM table2 WHERE condition);


SELECT columns
FROM table1
WHERE NOT EXISTS (SELECT 1 FROM table2 WHERE condition);
```